

TECHNICAL ARTICLE & RUNBOOK

PostgreSQL Indexing with Examples

*Selection, Design, Validation, Troubleshooting, and Production
Operations*

Validated against PostgreSQL 18.6 official documentation

Production baseline: PostgreSQL 18 | September 2026

Scope From access methods and operator classes through repeatable EXPLAIN tests, production-safe builds, monitoring, troubleshooting, and corrections to common indexing myths. Test all DDL with representative workload before production.

Table of Contents

1. Executive Summary	3
2. Fundamentals and Access Methods	3
3. Evidence-First Workflow	4
4. Reproducible Lab	4
5. B-tree and Hash	5
6. GIN: Text, Trigram, and JSONB	6
7. GiST, SP-GiST, and BRIN	7
8. Advanced Index Design	8
9. Production Lifecycle	9
10. Monitoring and Troubleshooting	9
11. Source Corrections	10
12. Decision Matrix and Pros/Cons	11
13. Production Checklist	12
14. Official Validation and References	12

1. Executive Summary

An index is useful only when its access method, operator class, key order, predicate, expression, and stored columns match the query. PostgreSQL still chooses the cheapest estimated plan. The existence—or even use—of an index does not guarantee a faster query.

Core method Capture the workload; baseline with EXPLAIN (ANALYZE, BUFFERS); design the smallest useful index; measure read, write, WAL, and storage effects; build safely; then monitor actual use.

Question	Required evidence
Predicates and operators	Exact SQL, parameters, collation, data types, operator classes.
Selectivity and skew	Actual rows, pg_stats, NULL fraction, common values, correlation.
Plan and I/O	EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS).
Write cost	DML latency, WAL, index size/count, HOT-update rate.
Production safety	Locks, disk/WAL headroom, replica lag, rollback.

2. Fundamentals and Access Methods

The heap and indexes are separate structures. Most index scans locate tuple identifiers and visit heap pages. An index-only scan avoids heap visits only when all needed columns are available from the index and the visibility map marks the page all-visible.

Method	Best fit	Key limitation
B-tree	Equality, ranges, ordered output.	Benefit declines as matched rows/pages rise.
Hash	Single-column equality.	No range, ordering, uniqueness, or multicolumn key.
GIN	tsvector, arrays, jsonb, trigrams.	Write amplification and pending-list work.
GiST	Ranges, geometry, nearest-neighbor, extensible search.	Behavior/lossiness depends on operator class.

Method	Best fit	Key limitation
SP-GiST	Partitioned point/prefix spaces, tries, quadtrees.	Coverage and distribution suitability vary.
BRIN	Huge, physically correlated tables.	Lossy page-range summaries.

No fixed maximum PostgreSQL has no “15 indexes per table” rule. Each index must justify read benefit against write, WAL, storage, vacuum, backup, and cache cost.

3. Evidence-First Workflow

1. Capture exact SQL, parameters, frequency, latency objective, concurrency, and returned columns.
2. Record a safe baseline with EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS, VERBOSE).
3. Inspect selectivity, skew, correlation, sizes, existing indexes, and statistics freshness.
4. Map predicates/order to access method, operator class, key order, predicate, expression, and INCLUDE.
5. Create one candidate and repeat the same workload under realistic data/cache/concurrency.
6. Compare plan, buffers, WAL, write latency, storage, and replica impact.
7. Build safely; monitor progress, blockers, capacity, and rollback.
8. Observe a full business cycle before removing overlapping or unused indexes.

```
EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS, VERBOSE)
SELECT * FROM lab.tickets WHERE passenger_id = '003861 803861';

SELECT attname, n_distinct, null_frac, most_common_vals,
       most_common_freqs, correlation
FROM pg_stats WHERE schemaname='lab' AND tablename='tickets';
```

4. Reproducible Lab

```
CREATE SCHEMA IF NOT EXISTS lab;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE TABLE lab.tickets (
    ticket_no bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    passenger_id text NOT NULL, passenger_name text NOT NULL,
    contact_data jsonb, created_at timestamptz NOT NULL
);
INSERT INTO lab.tickets(passenger_id,passenger_name,contact_data,created_at)
SELECT lpad((g%200000)::text,6,'0')||' '||lpad(g::text,6,'0'),
       (ARRAY['Alice','Bob','Viktor Belov','Sara'])[1+g%4],
       jsonb_build_object('phone','+'||(70000000000+g)::text,
                          'email','user'||g||'@example.test'),
       timestamptz '2025-01-01'+(g||' seconds')::interval
FROM generate_series(1,1000000) AS g;
ANALYZE lab.tickets;
```

Timings are not portable. Hardware, cache, row width, concurrency, selectivity, settings, extensions, and version alter results. Compare plans and buffers, not only elapsed milliseconds.

5. B-tree and Hash

5.1 B-tree

```
CREATE INDEX tickets_passenger_id_idx ON lab.tickets(passenger_id);
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM lab.tickets WHERE passenger_id='003861 803861';

CREATE INDEX tickets_created_desc_idx
ON lab.tickets(created_at DESC);
EXPLAIN (ANALYZE, BUFFERS)
SELECT ticket_no,created_at FROM lab.tickets
ORDER BY created_at DESC LIMIT 50;
```

B-tree is the general-purpose choice for scalar equality/range predicates and ordered scans. The operator class and collation must support the query semantics. A matching ordered index is especially valuable with LIMIT because it can stop early.

Complexity correction Do not promise $O(\log N)$ end-to-end CRUD. Duplicate matches, heap access, page splits, visibility, caching, WAL, locks, and returned-row count dominate observed cost.

5.2 Hash

```
CREATE INDEX tickets_passenger_hash_idx
ON lab.tickets USING hash(passenger_id);
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM lab.tickets WHERE passenger_id='003861 803861';
```

PostgreSQL 18 hash indexes are persistent and crash-recoverable. They store 4-byte hashes and can be smaller for long values, but support only equality, one key column, and no uniqueness. B-tree is generally recommended for concurrent scalar workloads because it has more features.

Myth correction Hash does not make the database query $O(1)$ or automatically faster. Collisions, bucket/page access, heap fetches, selectivity, cache, and concurrency still apply.

6. GIN: Text, Trigram, and JSONB

6.1 Full-text search

```
CREATE INDEX tickets_name_fts_idx ON lab.tickets
USING gin(to_tsvector('simple',passenger_name));
```

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT ticket_no FROM lab.tickets
WHERE to_tsvector('simple',passenger_name)
@@ plainto_tsquery('simple','viktor below');
```

The query expression and text-search configuration must match the index. PostgreSQL recommends GIN as the preferred full-text index; GiST text search uses lossy signatures and rechecks.

6.2 Trigram search

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX tickets_id_trgm_idx ON lab.tickets
USING gin(passenger_id gin_trgm_ops);
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM lab.tickets WHERE passenger_id ILIKE '%804386%';
```

pg_trgm GIN/GiST operator classes can serve LIKE, ILIKE, regex, equality, and similarity. Very short or pattern-free inputs can yield few/no useful trigrams; test real search terms.

6.3 JSONB

```
CREATE INDEX tickets_contact_ops_idx
ON lab.tickets USING gin(contact_data);           -- jsonb_ops
CREATE INDEX tickets_contact_path_idx
ON lab.tickets USING gin(contact_data jsonb_path_ops);
SELECT * FROM lab.tickets
WHERE contact_data @> '{"phone":"+70000000001"}';

CREATE INDEX tickets_phone_idx
ON lab.tickets((contact_data->>'phone'));
SELECT * FROM lab.tickets
WHERE contact_data->>'phone'='+70000000001';
```

Design	Operators / use	Trade-off
jsonb_ops	?, ? , ?&, @>, @?, @@	Flexible; generally larger.
jsonb_path_ops	@>, @?, @@	Often smaller/specific; no key-existence operators.
Expression B-tree	= or ordering on one extracted scalar	Small/targeted; only matching expression.

GIN write path One row can create many entries. fastupdate uses a pending list to reduce foreground cost, but merge work remains. Monitor latency, WAL, index size, pending list, and vacuum.

7. GiST, SP-GiST, and BRIN

7.1 GiST ranges and nearest-neighbor

```
CREATE TABLE lab.reservations(room_id int,during tstzrange NOT NULL);
CREATE INDEX reservations_during_gist_idx
ON lab.reservations USING gist(during);
SELECT * FROM lab.reservations
WHERE during && tstzrange(now(),now()+interval '2 hours');

CREATE TABLE lab.places(id bigint GENERATED ALWAYS AS IDENTITY,p point);
CREATE INDEX places_p_gist_idx ON lab.places USING gist(p);
SELECT * FROM lab.places ORDER BY p <-> point(500,500) LIMIT 10;
```

GiST is an extensible balanced framework; it is not simply an R-tree. SP-GiST supports partitioned, non-balanced structures such as tries, quadrees, and k-d trees. Choose by exact operator class, predicates, distribution, KNN support, and measurements—not a universal overlap rule.

Spatial correction A GiST index on PostgreSQL point does not serve a predicate that casts every value to PostGIS geometry/geography. Store and index a matching geometry/geography expression; validate SRID and distance units.

7.2 BRIN

```
CREATE INDEX tickets_created_brin_idx ON lab.tickets
USING brin(created_at) WITH (pages_per_range=64);
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*) FROM lab.tickets
WHERE created_at >= '2025-01-10' AND created_at < '2025-01-11';
SELECT attname,correlation FROM pg_stats
WHERE schemaname='lab' AND tablename='tickets';
```

BRIN stores compact summaries for physical page ranges. It is excellent for huge physically correlated tables and cheap to maintain, but is lossy and rechecks heap rows. Poor correlation can produce many false-positive ranges.

8. Advanced Index Design

8.1 Multicolumn keys and PostgreSQL 18 skip scan

```
CREATE INDEX tickets_name_created_idx
ON lab.tickets(passenger_name,created_at DESC);
SELECT ticket_no,created_at FROM lab.tickets
WHERE passenger_name='Alice'
ORDER BY created_at DESC LIMIT 20;
```

For B-tree, equality on leading keys and an inequality on the first non-equality key most tightly bound the scan. PostgreSQL 18 can use skip scan for a later-column condition when leading distinct values are

few enough; therefore “WHERE b can never use (a,b)” is incorrect. A dedicated (b) index may still be far better.

8.2 Covering indexes

```
CREATE INDEX tickets_lookup_cover_idx ON lab.tickets(passenger_id)
INCLUDE(ticket_no,passenger_name);
EXPLAIN (ANALYZE, BUFFERS)
SELECT ticket_no,passenger_name FROM lab.tickets
WHERE passenger_id='003861 803861';
```

INCLUDE columns are payload, not search keys, and do not participate in uniqueness. Wide payloads enlarge indexes. Index-only benefit also depends on all-visible heap pages maintained by VACUUM; inspect Heap Fetches.

8.3 Partial, expression, and unique

```
CREATE INDEX tickets_recent_idx ON lab.tickets(created_at)
WHERE created_at >= timestampz '2025-01-10';
CREATE UNIQUE INDEX tickets_email_uq
ON lab.tickets(lower(contact_data->>'email')) NULLS NOT DISTINCT;
ANALYZE lab.tickets;
```

A partial index is usable only when the planner can prove that the query predicate implies its predicate; implication is limited, and parameterized clauses often cannot prove it. Expression indexes add computation to writes. NULLS NOT DISTINCT makes nulls conflict in a unique index.

8.4 Multiple indexes versus composite

PostgreSQL can combine indexes with bitmap AND/OR, but bitmap combination normally loses ordering and needs a Sort. Composite indexes can win for stable filter/order patterns; separate indexes are more flexible. Measure and avoid redundant prefixes without workload evidence.

9. Production Lifecycle

Operation	Behavior	Use
CREATE INDEX	Reads continue; writes are blocked during build.	Approved maintenance window.
CREATE INDEX CONCURRENTLY	More work/scans and waits; avoids blocking writes.	Busy table with capacity monitoring.
REINDEX CONCURRENTLY	Reduced write blocking; restrictions apply.	Approved rebuild.
DROP INDEX CONCURRENTLY	Reduced blocking; restrictions apply.	Validated production removal.


```
-- Not inside BEGIN/COMMIT:
CREATE INDEX CONCURRENTLY tickets_passenger_prod_idx
ON public.tickets(passenger_id);
SELECT i.indexrelid::regclass,i.indisvalid,i.indisready
FROM pg_index i WHERE i.indrelid='public.tickets'::regclass;
DROP INDEX CONCURRENTLY IF EXISTS public.tickets_passenger_prod_idx;
```

Concurrent-build guardrails Only one concurrent index build can run on a table at a time. It cannot run inside a transaction block and can leave an INVALID index after failure. Monitor disk, WAL, replicas, long transactions, progress, and uniqueness failures. Investigate before retrying.

10. Monitoring and Troubleshooting

```
SELECT schemaname,relname,indexrelname,idx_scan,last_idx_scan,
       idx_tup_read,idx_tup_fetch
FROM pg_stat_user_indexes ORDER BY idx_scan,pg_relation_size(indexrelid) DESC;

SELECT pid,relid::regclass,index_relid::regclass,command,phase,
       lockers_total,lockers_done,blocks_total,blocks_done,
       tuples_total,tuples_done
FROM pg_stat_progress_create_index;
```

Statistics are cumulative and may reset. `idx_scan=0` does not prove an index is safe to drop: account for replicas, seasonal jobs, constraints, prepared statements, failover requirements, and the observation window.

Symptom	Checks	Corrective direction
Seq scan despite index	Selectivity, estimates, casts, expression, collation/opclass.	ANALYZE; fix mismatch; redesign; or accept cheaper seq scan.
Index used but slow	Actual rows, heap blocks, I/O, sort, rechecks.	Improve selectivity/covering/order; return fewer rows.
Bitmap touches most pages	Heap Blocks versus table pages.	Index may add little; partial/composite or seq scan.
Index-only has heap fetches	Heap Fetches, visibility map, vacuum.	Restore vacuum cadence; do not force.
GIN slow	Terms, opclass, pending list, rechecks.	Match query/opclass; maintain or redesign.
BRIN lossy	Correlation, pages_per_range.	Improve order; smaller ranges; B-tree.
Concurrent build waits	Progress, lockers, old transactions.	Resolve confirmed blockers; schedule.

Symptom	Checks	Corrective direction
Writes/WAL rise	Index count/width, HOT rate, replicas.	Remove proven redundancy; narrow/partial design.
Bad ORDER BY plan	Direction, NULL order, filter/key order.	Matching B-tree or accept Sort.
Bad estimates	pg_stats, skew, cross-column correlation.	ANALYZE; statistics target; CREATE STATISTICS.

11. Source Corrections

Source claim	PostgreSQL 18 validated correction
50k/1M rows define performance.	No row-count threshold; workload, width, cache, selectivity, storage, and concurrency dominate.
Maximum 15 indexes.	Not a PostgreSQL rule; use measured economics.
Hash CRUD is $O(1)$ and significantly faster.	Unsupported end-to-end guarantee; hash is equality-only and workload dependent.
GIN is a B-tree with $O(\log N)$.	GIN is an inverted index with posting structures; the simplification is misleading.
GiST is an R-tree.	GiST is a generalized balanced framework; some opclasses provide R-tree-equivalent behavior.
Overlap decides GiST versus SP-GiST.	Too broad; operator class, data, predicates, KNN, and measurements decide.
(a,b) cannot serve WHERE b.	PostgreSQL 18 skip scan can sometimes use later-column constraints.
Two nulls are always distinct in unique indexes.	Default is distinct; NULLS NOT DISTINCT changes semantics.
GiST(point) serves casted PostGIS geography.	The indexed expression must match the geometry/geography predicate.
Example milliseconds prove benefit.	Timings are environment-specific; reproduce with EXPLAIN and buffers.

12. Decision Matrix and Pros/Cons

Requirement	Start with	Caveat
Scalar equality/range	B-tree	General-purpose; selectivity matters.
Long scalar equality only	B-tree; benchmark hash	Hash loses range/order/unique/multicolumn.
JSON containment/path	GIN	Choose opclass by operators; write cost.
Full-text search	GIN tsvector	Expression/configuration must match.
Substring/regex	pg_trgm GIN/GiST	Extension; short patterns/write cost.
Ranges/overlap/exclusion	GiST	Opclass behavior; possible recheck.
Partitioned point/prefix	SP-GiST	Verify coverage and distribution.
Huge ordered table	BRIN	Requires physical correlation.
Filter + ordered LIMIT	Composite B-tree	Key order is workload-specific.
Narrow hot subset	Partial	Predicate implication matters.
Read-mostly covering	B-tree INCLUDE	Visibility and index width matter.

13. Production Checklist

- ☐ Exact PostgreSQL 18.x release and service restrictions recorded.
- ☐ Top SQL, parameters, frequency, objective, and returned columns captured.
- ☐ Safe representative baseline retained with EXPLAIN ANALYZE/BUFFERS/WAL.
- ☐ Statistics, skew, correlation, sizes, and index overlap reviewed.
- ☐ Method, opclass, collation, expression, predicate, and key order verified.
- ☐ Read benefit compared with writes, WAL, storage, vacuum, backup, and replicas.
- ☐ Build method, lock behavior, headroom, monitoring, and rollback approved.
- ☐ Post-build plan and business latency validated; ANALYZE used where appropriate.
- ☐ Full workload cycle observed before removal.
- ☐ Constraint dependencies and replica usage checked before drop.
- ☐ Owner, purpose, creation SQL, query fingerprints, and review date documented.

14. Official Validation and References

PostgreSQL 18.6 official documentation is the primary authority. The LinkedIn article supplied examples and topic direction; its fixed timings and simplified rules were not treated as guarantees. Revalidate syntax, extensions, privileges, and behavior against the deployed release.

- Indexes: <https://www.postgresql.org/docs/18/indexes.html>
- Index Types: <https://www.postgresql.org/docs/18/indexes-types.html>
- Multicolumn Indexes: <https://www.postgresql.org/docs/18/indexes-multicolumn.html>
- Indexes and ORDER BY: <https://www.postgresql.org/docs/18/indexes-ordering.html>
- Combining Indexes: <https://www.postgresql.org/docs/18/indexes-bitmap-scans.html>
- Unique Indexes: <https://www.postgresql.org/docs/18/indexes-unique.html>
- Expression Indexes: <https://www.postgresql.org/docs/18/indexes-expressional.html>
- Partial Indexes: <https://www.postgresql.org/docs/18/indexes-partial.html>
- Index-Only Scans: <https://www.postgresql.org/docs/18/indexes-index-only-scans.html>
- Examining Index Usage: <https://www.postgresql.org/docs/18/indexes-examine.html>
- CREATE INDEX: <https://www.postgresql.org/docs/18/sql-createindex.html>
- Progress Reporting: <https://www.postgresql.org/docs/18/progress-reporting.html>
- pg_trgm: <https://www.postgresql.org/docs/18/pgtrgm.html>
- JSONB Indexing: <https://www.postgresql.org/docs/18/datatype-json.html#JSON-INDEXING>
- Text Search Indexes: <https://www.postgresql.org/docs/18/textsearch-indexes.html>
- Access Methods: <https://www.postgresql.org/docs/18/indextypes.html>
- Source Article: <https://www.linkedin.com/pulse/postgresql-indexing-examples-firdaus-al-ghifari/>

Optional lab cleanup: DROP SCHEMA lab CASCADE;